
Inductive Bias, Random Features, and Transfer on LSST: A Compact Comparison of Logistic Regression, ROCKET, and TSPulse

Mathis Vehils-Vinals Edgar Duc Colin Laurent

M2DS (ID=20) – https://github.com/Vehils-Vinals/time_series_lsst

1. Introduction

We study multivariate time-series classification on LSST from the UEA/UCR archive. The repository compares three paradigms: a linear baseline on flattened inputs, ROCKET (2; 3; 4) as a random convolutional feature map, and TSPulse (5) as a pre-trained foundation model for time series. The main issue is the trade-off between inductive bias, computational cost, and robustness under severe class imbalance. In this regime, accuracy and macro-F1 need not agree: the former is dominated by major classes, whereas the latter weights classes uniformly.

2. Theoretical Perspective

The LSST dataset consists of simulated astronomical light curves observed in six spectral bands. Each sample is a multivariate series $x \in \mathbb{R}^{36 \times 6}$, and the task is 14-way classification. Since the repository only provides numeric labels, without astrophysical semantics, the analysis is representation-centric.

Flattened linear baseline. The baseline vectorizes the signal as $z = \text{vec}(x) \in \mathbb{R}^{216}$ and fits multinomial logistic regression:

$$p(y = k | x) = \text{softmax}_k(Wz + b).$$

Its strengths are convex optimization, negligible cost, and moderate interpretability. Its weakness is structural: flattening removes explicit temporal locality, provides no shift-robust motif extraction, and cannot compose local patterns hierarchically. The task is therefore reduced to linear separation in a coordinate system poorly aligned with temporal discrimination.

ROCKET as random temporal features. ROCKET maps a signal to a large random feature representation

$$x \mapsto \phi(x) \in \mathbb{R}^m, \quad p(y = k | x) = \text{softmax}_k(A\phi(x) + c),$$

where $\phi(x)$ is built from many random convolutions and simple response summaries. This injects temporal locality without expensive end-to-end feature learning: local motifs become easier to separate in a high-dimensional embedding,

while the learned classifier remains linear. ROCKET thus offers a strong accuracy–efficiency compromise.

TSPulse as transfer learning. TSPulse can be summarized as

$$x \mapsto h_\theta(x), \quad p(y = k | x) = \text{softmax}_k(Uh_\theta(x) + d),$$

where h_θ is largely inherited from pre-training and only a small subset of parameters is adapted to LSST. The appeal is the reuse of generic temporal representations that may encode local motifs, channel interactions, and broader structure before task-specific training. The cost is a much heavier and more protocol-sensitive optimization process.

Metric tension under imbalance. The three methods define a clean hierarchy of biases: no explicit temporal structure for the baseline, random local motifs for ROCKET, and transferred hierarchical representations for TSPulse. Under imbalance, the metric is part of the scientific question:

$$\text{Acc} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\hat{y}_i = y_i\}, \quad \text{MacroF1} = \frac{1}{K} \sum_{k=1}^K \frac{2P_k R_k}{P_k + R_k}.$$

Accuracy is frequency-weighted, whereas macro-F1 penalizes failures on rare classes much more strongly.

3. Experimental Protocol

The observed LSST split is train (2459, 36, 6), test (2466, 36, 6), with 14 classes. The class distribution is highly imbalanced, ranging from 7 to 777 examples per class. Because label semantics are undocumented in the repository, the discussion is restricted to discriminative structure rather than astrophysical interpretation.

The baseline uses `vec(x)`, `StandardScaler`, and `LogisticRegression`. ROCKET uses the `aeon` implementation with the `MiniRocket` variant. TSPulse loads the IBM Granite checkpoint from `tsfm-public`, and fine-tunes a classification pipeline with a largely frozen backbone. Global hyperparameters are seed 42, batch size 32, learning rate 10^{-3} , context length 512, and a maximum of 50 epochs. TSPulse uses a 90/10 split inside the

Table 1. Test performance and training cost from artifacts/evaluations.

Model	Accuracy	Macro-F1	Train time
LogisticRegression	0.2044	0.1510	1.49 s
ROCKET	0.6521	0.5152	20.51 s
TSPulse	0.6334	0.4645	2091.53 s

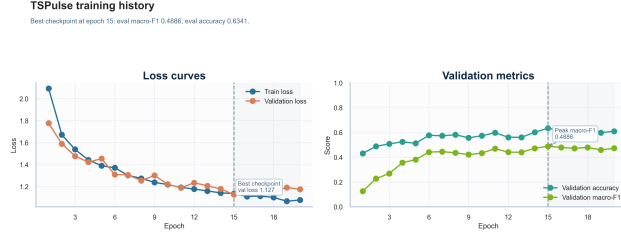


Figure 1. TSPulse training history from the repository. Validation macro-F1 peaks around epoch 15, supporting the early-stopping checkpoint used for test evaluation.

training set for validation, with early stopping patience 5 on `eval_macro_f1`. The best TSPulse checkpoint is obtained at epoch 15. The protocol is therefore slightly asymmetric, since logistic regression and ROCKET use the full official training split.

Figure 1 is useful because it shows that fine-tuning is not monotonically improving: the best validation trade-off is reached relatively early, which is consistent with partial fine-tuning on a small and imbalanced dataset.

4. Results and Discussion

The baseline is decisively too weak: an accuracy of 0.2044 and macro-F1 of 0.1510 show that LSST is not close to linearly separable in flattened coordinates. ROCKET reaches the best overall accuracy (0.6521) while remaining inexpensive (20.51 s), which supports the view that random local temporal filters can naturally match the structure of short, motif-driven signals.

TSPulse is slightly behind in accuracy (0.6334), and in macro-F1 (0.4645). Moreover, the cost is substantial: 2091.53 s, roughly 100× the ROCKET cost (Figure 2). This suggests that the most critical factor for LSST classification is not compute and memory, but rather feature extraction, where ROCKET is more aligned with the dataset scale. 36-time-steps sequences are very short for foundation models. However, we can say that TSPulse is able to deduce very discriminative decision boundaries from raw features. It might also be that the TSPulse pre-training was done on datasets and features that do not resemble LSST, so that we might need to re-train the whole architecture for ideal results.

The class-wise artifacts refine this picture. Classes 52,

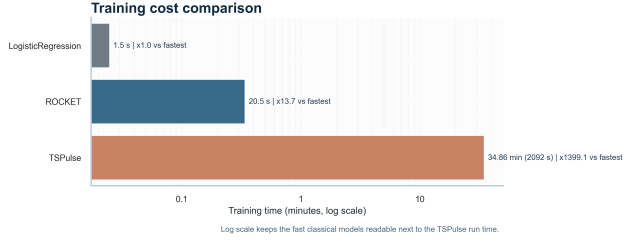


Figure 2. Training-time comparison from the repository. TSPulse is vastly more expensive than both the linear baseline and ROCKET.

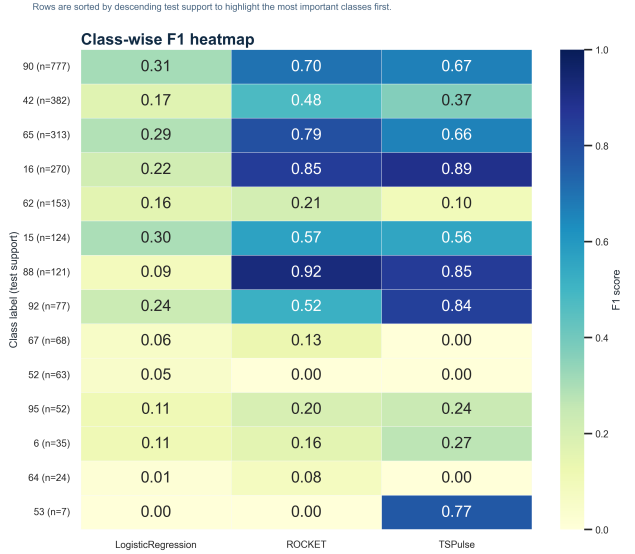


Figure 3. Class-wise F1 heatmap from the repository. TSPulse achieves a more balanced per-class profile, while ROCKET remains strongest on several high-support classes.

64, and 67 remain difficult across methods. By contrast, TSPulse recognizes the rare class 53 well, unlike the two other approaches. This remains suggestive rather than definitive because the support is very small, but it is consistent with the hypothesis that transferred representations may preserve minority-class structure better than either a raw linear model or random features alone. The repository also contains additional visualizations: the score overview is consistent with Table 2, the confusion matrices are denser but less compact, and the ROCKET confidence distribution is degenerate in the saved artifacts.

Limits. The study is exploratory rather than definitive. Only one visible seed is reported, with no confidence intervals or multi-seed repetition. The exact runtime environment is only partially documented. The training protocol is not perfectly symmetric because TSPulse uses an internal validation split. TSPulse training includes GPU-parallelized computations, whereas ROCKET and logistic regression are CPU-based.

5. Conclusion

The comparison separates three regimes clearly. Logistic regression is a useful minimal baseline but structurally insufficient. ROCKET delivers the best compromise between accuracy and computational cost by injecting a strong yet cheap temporal inductive bias. TSPulse yields solid accuracy and macro-F1, and may be more sensitive to minority classes, but at a much larger optimization cost. The most natural next steps are multi-seed evaluation, heavier fine-tuning or even full training on LSST classification.

References

- [1] M. Vehils-Vinals, *time_series_lsst*. GitHub repository, https://github.com/Vehils-Vinals/time_series_lsst, 2026.
- [2] A. Dempster, F. Petitjean, and G. I. Webb, *ROCKET: Exceptionally Fast and Accurate Time Series Classification Using Random Convolutional Kernels*. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2020.
- [3] A. Dempster, D. F. Schmidt, and G. I. Webb, *MiniRocket: A Very Fast (Almost) Deterministic Transform for Time Series Classification*. KDD 2021.
- [4] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, *MultiRocket: Multiple pooling operators and transformations for fast and effective time series classification*. Data Mining and Knowledge Discovery, 2022.
- [5] V. Ekambaram, S. Kumar, A. Jati, S. Mukherjee, T. Sakai, P. Dayama, W. M. Gifford, and J. Kalagnanam, *TSPulse: Dual Space Tiny Pre-Trained Models for Rapid Time-Series Analysis*. arXiv preprint arXiv:2505.13033, 2025.

A. Appendix : Additional Figures

A.1. Dataset analysis

LSST training dataset contains 2705 examples of time series. Each time series has 36 time steps and 6 bands. The test set contains 2466 examples (with again 36 time steps and 6 bands).

The examples are sorted into 14 highly-imbalanced classes. The class distributions of training and test sets are reported in figures 5 and 6.

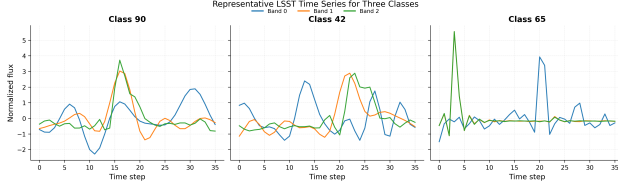


Figure 4. One time series representative for 3 different classes.

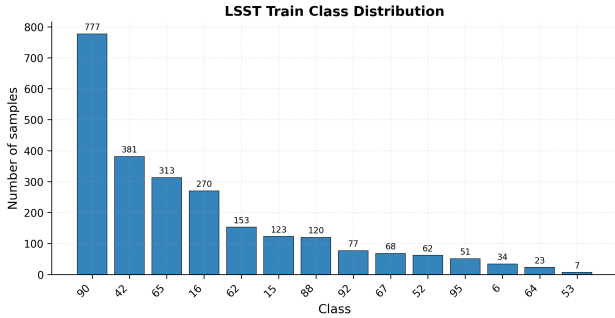


Figure 5. Train class distribution.

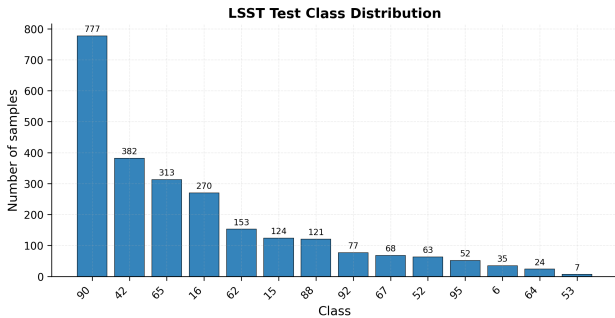


Figure 6. Test class distribution.

A.2. Full results

We report here some other promising numerical experiments conducted across models and configurations (hyperparameters).

For all ROCKET variants, we used 10000 kernels and 4 features per kernel.

For TSPulse, we directly import `TSPulseForClassification` from the IBM Granite *tsfm-public* repository, keep backbone frozen and perform fine-tuning of classification head only. We add for each original channel a new "first-order difference" channel. We then normalize each channel with Z-score. We use linear interpolation to transform the 36-time-steps series into the expected 512-time-steps sequences. We have 97K trainable parameters out of 0.38M.

For MOMENT, we directly import the `MOMENTPipeline` from the *momentfm* library, with the built-in classification head. We use same setup as TSPulse for fair comparison. We have 0.13M trainable parameters out of 35.5M. The MOMENT training is roughly 6 times slower than TSPulse. In the *config* column, we use the following codes : LR : learning rate (default 0.001), DRP : dropout (default 0.1), MR : mask ratio (default 0.2), CLS : classification mode (*full embedding* or *time with short FFT embedding*, default FULL).

In our experiments, we find that basic cross-entropy loss is always better than class-weighted cross-entropy or focal loss. We also tried a class-weighted sampler, with no positive effect. Adding more artificial features has no positive effect.

Table 2. Test performance for all models and configurations

Model	Config	Accuracy	Macro-F1
LogisticRegression	-	0.2044	0.1510
MiniRocket (+Logistic)	-	0.6521	0.5152
MultiRocket (+Ridge)	-	0.6443	0.4007
TSPulse	<i>default</i>	0.6334	0.4645
TSPulse	CLS : FFT	0.6451	0.4419
TSPulse	DRP : 0.15	0.6350	0.4611
TSPulse	MR : 0.3	0.6320	0.4477
MOMENT	<i>default</i>	0.5989	0.3659